# Reverse proxies and load balancers

Run nginx, HAProxy and Envoy in front of services: routing, TLS, load balancing, timeouts, caching, rate limits and debugging 502 and 504 errors.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

## Contents

1. What a reverse proxy does (12 min)
2. nginx configuration: servers and locations (13 min)
3. Proxying to upstreams: headers, buffers and WebSockets (13 min)
4. TLS termination, HTTP/2 and certificates (13 min)
5. Load balancing: algorithms and health checks (13 min)
6. Timeouts, retries, keepalive and connection limits (13 min)
7. Caching, compression and static files (12 min)
8. Rate limiting and protecting backends (13 min)
9. HAProxy: frontends, backends and ACLs (13 min)
10. Envoy, Traefik and Caddy: dynamic proxies (13 min)
11. Zero-downtime reloads and blue-green at the proxy (12 min)
12. Debugging 502, 504 and other proxy problems (14 min)
Cheat sheet

# Lesson 1. What a reverse proxy does

12 minutes. Learn what sits between users and your services, the jobs a reverse proxy does and how it differs from a forward proxy and a load balancer.

Almost every production web service sits behind a proxy. It is the front door: it decides which backend gets each request and enforces rules before the traffic reaches your code. Understanding its jobs makes every later topic, from TLS to timeouts, easier.

## What you will learn

- The difference between forward proxies, reverse proxies and load balancers
- The jobs a reverse proxy does
- Layer 4 and layer 7 proxying
- Where the proxy sits in an architecture

## Forward, reverse and load balancer

A **forward proxy** sits on the client side and sends requests out for users (for example a company web filter). A **reverse proxy** sits on the server side and receives requests for your services. A **load balancer** is a reverse proxy whose main job is to spread requests across several backends. In practice one program, such as nginx, HAProxy or Envoy, often does all three roles.

## Jobs of a reverse proxy

- **Routing:** send `api.example.com` to one service and `/static/` to another.
- **TLS termination:** handle certificates and encryption once, speak plain or re-encrypted traffic inside.
- **Load balancing and health checks:** share requests and stop sending to failed backends.
- **Protection:** rate limits, size limits, request filtering and hiding backend details.
- **Performance:** compression, connection reuse and caching of responses.
- **Observability:** one place to log every request.

## Layer 4 and layer 7

A **layer 4** proxy forwards TCP or UDP streams without reading them, which is fast and works for any protocol. A **layer 7** proxy understands HTTP, so it can route by host, path, header or cookie and can rewrite requests. Choose layer 4 for databases and raw TLS passthrough, layer 7 for web traffic.

### Diagram: A reverse proxy in front of services

| Part | What it does | What goes wrong |
|---|---|---|
| **Users** | Browsers and apps that connect to one public address. | If the address changes without DNS updates, users cannot reach you. |
| **Reverse proxy** | Terminates TLS, applies rules and picks a backend. | A single proxy is a single point of failure; run at least two. |
| **Web service** | Serves pages for the main host. | If it fails, the proxy should stop routing to it. |
| **API service** | Serves API paths on private addresses. | Exposed directly, it skips the proxy's protections. |
| **Access logs** | Every request is recorded at the front door. | Without logs you cannot tell users from attackers. |

Flow: Users to Reverse proxy / Reverse proxy to Web service / Reverse proxy to API service / Reverse proxy to Access logs

MARGA by EknathaLabs marga.eknathalabs.com

## What a proxy changes

Backends now see the proxy's address, not the user's. The proxy passes the original address in headers such as `X-Forwarded-For`, and the application must trust that header only from the proxy. The proxy is also now a place where timeouts, buffer sizes and limits can break otherwise healthy requests, which later lessons cover.

## Incident: Three ports, three certificates

> **Incident:** *A team exposed each app on its own public port and one certificate expired unnoticed.*

```
$ curl -I https://shop.example.com:8443
curl: (60) SSL certificate problem: certificate has expired
$ ss -ltn | grep -c LISTEN
11
```

They moved to one proxy on port 443 with automatic certificate renewal, closed the app ports and gained one access log. Expiry alerts moved from three places to one.

## Common mistakes

- Exposing backends directly as well as through the proxy
- Trusting X-Forwarded-For from the whole internet
- Running a single proxy with no standby
- Using layer 7 rules for a protocol the proxy does not understand

### Check yourself

**What is a reverse proxy?**

Answer: A server that receives requests on behalf of backends

**Which proxy type reads HTTP and routes by path?**

Answer: Layer 7

**Why does a backend see the proxy's address?**

Answer: The proxy opens its own connection to the backend

**Why run two proxies?**

Answer: To avoid a single point of failure

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 2. nginx configuration: servers and locations

13 minutes. Read and write nginx configuration: contexts, server blocks, location matching rules and how to test and reload safely.

nginx configuration looks simple and hides a few rules that cause most mistakes. Once you know how a server is chosen and how locations are matched, you can read any nginx file and predict what it does.

## What you will learn

- Contexts and directives in nginx.conf
- How server and server_name are chosen
- Location matching order
- Testing and reloading safely

## Contexts

```
events { worker_connections 1024; }
http {
  include mime.types;
  server {
    listen 80;
    server_name shop.example.com;
    root /var/www/shop;

    location / { try_files $uri $uri/ /index.html; }
    location /api/ { proxy_pass http://127.0.0.1:8080; }
    location = /healthz { return 200 "ok\n"; }
    location ~* \.(png|jpg|css|js)$ { expires 7d; }
  }
}
```

`events` and `http` are top-level contexts. `server` lives in `http`, `location` in `server`. A directive set in an outer context is inherited unless an inner block sets it again.

## Choosing the server

nginx matches the `listen` address and port first, then compares the request's `Host` header with `server_name`. If nothing matches it uses the default server for that port (the first one, or the one marked `default_server`). A catch-all default that returns 444 or 404 stops unknown hostnames from reaching a real site.

## Location matching

| Form | Meaning | Priority |
|---|---|---|
| `location = /x` | Exact match | Highest, stops searching |
| `location ^~ /x` | Prefix, skip regexes | Stops regex checks |
| `location ~ re` / `~* re` | Regex (case sensitive or not) | Checked in file order |
| `location /x` | Prefix | Longest prefix remembered, lowest |

nginx finds the longest matching prefix, then checks regexes in order, and a matching regex wins over a plain prefix. That is why a broad image regex can capture `/api/photo.png`.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How nginx picks where a request goes

| Part | What it does | What goes wrong |
|---|---|---|
| **Request** | Arrives with an address, port, Host header and path. | A missing Host header falls back to the default server. |
| **Server block** | Chosen by listen and server_name. | No default_server means the first block answers unknown hosts. |
| **Location** | Exact, then prefix, then regex rules choose a location. | A broad regex can steal paths from a prefix. |
| **Action** | Serve a file, proxy_pass, return or redirect. | A wrong proxy_pass trailing slash rewrites the path. |
| **nginx -t** | Checks the syntax before reload. | Reloading without a test can take the site down. |

Flow: Request to Server block / Server block to Location / Location to Action / nginx -t to Server block

## Test, then reload

`nginx -t` reports syntax and file errors. `nginx -s reload` starts new workers with the new configuration and lets old ones finish their requests, so no connection is dropped. If the test fails, the old configuration keeps running.

## Incident: The regex that stole the API

**Incident:** *A new image location used a regex that also matched paths under /api/.*

```
$ nginx -t
nginx: configuration file /etc/nginx/nginx.conf test is successful
$ curl -sI https://shop.example.com/api/photo.png | head -1
HTTP/1.1 404 Not Found
```

The syntax was valid, so the test passed. The request matched the regex location before the /api/ prefix, and was served from disk. Adding ^~ to the API location stopped regex matching for that prefix.

## Common mistakes

- Assuming the first location in the file wins
- Reloading without running nginx -t
- Having no default server for unknown hosts
- Repeating the same directive in every block instead of using inheritance

### Check yourself

**Which location form is checked first and stops the search?**

Answer: Exact match with =

**What does ^~ do?**

Answer: Stops regex matching after a prefix match

**What does nginx -t do?**

Answer: Checks the configuration without applying it

**What chooses the server block?**

Answer: listen address and server_name

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 3. Proxying to upstreams: headers, buffers and WebSockets

13 minutes. Forward requests to backends with proxy_pass, pass the right headers, understand buffering and support WebSockets correctly.

A proxy that simply passes bytes along is not enough. Applications use the Host header, the client address and the scheme to build links, set cookies and log users. Getting these headers right, and understanding buffering, prevents a long list of confusing bugs.

## What you will learn

- proxy_pass and how paths are rewritten
- Headers the backend needs
- Buffering, streaming and large uploads
- Proxying WebSockets and server-sent events

## proxy_pass and the trailing slash

```
location /api/ {
    proxy_pass http://app_backend;          # sends /api/users as /api/users
}
location /v1/ {
    proxy_pass http://app_backend/;         # sends /v1/users as /users (prefix replaced)
}
```

With no URI after the host, the original path is sent. With a URI (even just `/`), the matched prefix is replaced. One character changes what the backend receives.

## Headers

```
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

The app should trust these headers only when the request comes from the proxy's address. Otherwise anyone can claim to be any client by sending the header.

## Buffering

By default nginx reads the whole response from the backend into buffers and then sends it at the client's pace, which frees the backend quickly. For streaming responses and server-sent events turn it off with `proxy_buffering off;`. Large uploads are limited by `client_max_body_size` (default 1 MB), which returns 413 when exceeded.

## WebSockets

```
map $http_upgrade $connection_upgrade { default upgrade; '' close; }
location /ws/ {
    proxy_pass http://app_backend;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $connection_upgrade;
    proxy_read_timeout 3600s;
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: What the proxy tells the backend

| Part | What it does | What goes wrong |
|---|---|---|
| **Client** | Sends Host, cookies and its own address. | The client can also forge headers such as X-Forwarded-For. |
| **nginx** | Rewrites the request and sets trusted forwarding headers. | If it forgets Host, the app builds wrong links. |
| **Backend app** | Reads Host, scheme and client address from headers. | Trusting headers from anyone but the proxy is a spoofing hole. |
| **WebSocket** | A long-lived upgraded connection through the same proxy. | Default timeouts close idle sockets after 60 seconds. |

Flow: Client to nginx / nginx to Backend app / nginx to WebSocket

## Incident: Redirects to localhost

**Incident:** *After moving behind nginx a login redirected users to http://localhost:3000/dashboard.*

```
$ curl -sI https://shop.example.com/login
HTTP/1.1 302 Found
Location: http://localhost:3000/dashboard
```

The app built the absolute URL from the Host and scheme it received, which were the proxy's connection to localhost. Passing Host and X-Forwarded-Proto and telling the app to trust the proxy fixed the redirect.

## Common mistakes

- Changing proxy_pass without noticing the trailing-slash rule
- Leaving the default 1 MB upload limit on an upload endpoint
- Forgetting WebSocket upgrade headers and timeouts
- Letting any client send X-Forwarded-For that the app trusts

### Check yourself

**What does a URI in proxy_pass (even /) do?**

Answer: Replaces the matched location prefix

**Which header carries the original scheme?**

Answer: X-Forwarded-Proto

**Why turn proxy_buffering off for some responses?**

Answer: For streaming and server-sent events

**What status appears when an upload exceeds client_max_body_size?**

Answer: 413

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 4. TLS termination, HTTP/2 and certificates

13 minutes. Terminate TLS at the proxy with sound protocols, redirects, HSTS, OCSP stapling and automated certificate renewal.

The proxy is where users meet TLS. A good setup is short: a valid chain, modern versions, a redirect from plain HTTP and renewal that nobody has to remember. A bad one fails on a date you cannot see.

## What you will learn

- A minimal HTTPS server block
- Protocols, ciphers and HTTP/2
- HSTS, redirects and OCSP stapling
- Automated renewal and re-encryption to backends

## A minimal HTTPS server

```
server { listen 80; server_name shop.example.com; return 301 https://$host$request_uri; }
server {
    listen 443 ssl;
    http2 on;
    server_name shop.example.com;
    ssl_certificate     /etc/ssl/shop/fullchain.pem;
    ssl_certificate_key /etc/ssl/shop/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_session_cache shared:SSL:10m;
    add_header Strict-Transport-Security "max-age=31536000" always;
    location / { proxy_pass http://app_backend; }
}
```

Use the **full chain** (your certificate plus intermediates). A missing intermediate works in some browsers and fails in others and in `curl`. Keep the private key readable only by root.

## Modern settings

TLS 1.2 and 1.3 are enough; older versions are retired. HTTP/2 multiplexes many requests over one connection and is enabled with `http2 on`. **HSTS** tells browsers to use HTTPS only for a period, so start with a short value before committing to a year. **OCSP stapling** lets the server include proof that the certificate is not revoked, which speeds up connections.

## Renewal

Most public certificates come from an ACME service. A client such as certbot or acme.sh proves control of the domain, fetches a certificate and runs a hook to reload the proxy. Test renewal with a dry run, alert at 21 days to expiry, and monitor the certificate the proxy actually serves, not just the file on disk.

### Diagram: Certificate renewal that does not depend on a person

| Part | What it does | What goes wrong |
|---|---|---|
| **ACME client** | Requests and renews certificates automatically. | If the job fails silently, expiry arrives unnoticed. |
| **Certificate files** | fullchain.pem and a protected private key on the proxy. | A wrong chain order breaks non-browser clients. |
| **Proxy reload** | A deploy hook reloads nginx so it serves the new file. | Renewed on disk but not reloaded still serves the old one. |
| **Users** | Browsers and clients see the live certificate. | An expired certificate blocks access completely. |
| **Expiry monitor** | Checks the served certificate and alerts early. | No monitor means the first alert is a customer. |

MARGA by EknathaLabs marga.eknathalabs.com

Flow: ACME client to Certificate files / Certificate files to Proxy reload / Proxy reload to Users / Expiry monitor to Proxy reload

## Re-encrypting to backends

If the network between proxy and app is not fully trusted, use `proxy_pass https://...` and verify the backend certificate with `proxy_ssl_verify on`. Otherwise plain HTTP inside a private network is common, but the choice should be conscious and written down.

## Incident: The certificate that renewed but did not reload

**Incident:** *A renewal ran on schedule but the proxy kept serving the old certificate until it expired.*

```
$ echo | openssl s_client -connect shop.example.com:443 2>/dev/null | openssl x509 -noout -enddate
notAfter=Nov  2 03:00:00 2026 GMT
$ ls -l /etc/ssl/shop/fullchain.pem
-rw-r--r-- 1 root root 5412 Oct 28 03:00 fullchain.pem
```

The file on disk was new but nginx had never reloaded. The team added a post-renewal hook that runs nginx -t and reload, and a monitor that checks the live certificate's expiry.

## Common mistakes

- Serving the leaf certificate without the intermediates
- Checking only the file on disk, not the served certificate
- Setting a one-year HSTS before the site is fully on HTTPS
- Leaving old TLS versions enabled for no reason

### Check yourself

**What should ssl_certificate point to?**

Answer: The full chain file

**What does HSTS do?**

Answer: Tells browsers to use HTTPS only

**What must happen after a certificate is renewed on disk?**

Answer: The proxy must reload to serve it

**What is best to monitor?**

Answer: The certificate the proxy actually serves

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 5. Load balancing: algorithms and health checks

13 minutes. Spread traffic across backends with round robin, least connections and hashing, and remove failed backends using passive and active health checks.

Spreading requests is easy. Doing it well means choosing a method that fits your traffic and, more importantly, knowing when a backend is sick and taking it out of rotation quickly and safely.

## What you will learn

- Round robin, weighted, least connections and hashing
- Session stickiness and when to avoid it
- Passive and active health checks
- Draining, slow start and retries

## Algorithms

```
upstream app_backend {
    least_conn;
    server 10.0.1.11:8080 weight=2 max_fails=3 fail_timeout=15s;
    server 10.0.1.12:8080;
    server 10.0.1.13:8080 backup;
}
```

| Method | Good for |
|---|---|
| Round robin | Equal servers and short, similar requests |
| Weighted | Servers of different size |
| Least connections | Long or uneven requests |
| Hash on a key (`hash $request_uri consistent`) | Caches, where the same key should reach the same server |
| `ip_hash` | Simple stickiness by client address |

## Stickiness

Sending a user to the same backend helps when sessions live in server memory, but it spreads load unevenly and a failed server loses its users' sessions. Prefer to keep session data in a shared store and balance freely.

## Health checks

**Passive** checks watch real traffic: after `max_fails` errors within a time, the server is skipped for `fail_timeout`. **Active** checks (HAProxy's `check`, nginx Plus or an external checker) probe an endpoint such as `/healthz` on a schedule. A good health endpoint checks the things the service needs (database, queue) but is cheap and does not fail because of a slow optional dependency.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: Balancing with health checks

| Part | What it does | What goes wrong |
|---|---|---|
| **Load balancer** | Chooses a backend using the algorithm. | A bad algorithm for the traffic loads one server heavily. |
| **Backend A** | Healthy and in rotation. | If it fails, passive checks find out only after users see errors. |
| **Backend B** | Hung but listening; its health check fails. | A port-only check keeps sending it traffic. |
| **Health check** | Probes a health endpoint on a schedule. | A shallow check says healthy when the service is broken. |
| **Drain** | Stops new requests, lets current ones finish. | Killing a server mid-request drops user requests. |

Flow: Load balancer to Backend A / Load balancer to Backend B / Health check to Backend B / Load balancer to Drain

## Changing servers safely

Before updating a backend, **drain** it: stop new requests and wait for current ones to finish. When adding a server, **slow start** ramps its share up so a cold cache does not get a full load at once. Limit retries to idempotent requests so a retry does not charge a card twice.

## Incident: One in four requests failed

**Incident:** *A hung backend kept passing a TCP check and received a quarter of the traffic.*

```
$ for i in $(seq 20); do curl -s -o /dev/null -m 3 -w '%{http_code} ' https://lb.example.com/api;  \
    done
200 200 504 200 200 200 504 200 200 504 200 200 ...
backend B: tcp check ok, /healthz timeout
```

The load balancer only checked that port 8080 accepted a connection. Switching to an HTTP check of /healthz removed the hung server within seconds, and an alert now fires when any backend leaves rotation.

## Common mistakes

- Health checks that only test the TCP port
- Sticky sessions to hide state kept in memory
- Retrying non-idempotent requests
- Removing a server without draining it

### Check yourself

**Which method suits long, uneven requests?**

Answer: Least connections

**What does a passive health check use?**

Answer: Real traffic errors

**What is draining?**

Answer: Finishing current requests before removing a server

**Why avoid retries for non-idempotent requests?**

Answer: A retry could repeat the action

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 6. Timeouts, retries, keepalive and connection limits

13 minutes. Set proxy timeouts that match real behaviour, tune keepalive, and understand how retries and limits cause or prevent outages.

Most proxy outages are not crashes but timers: a read timeout shorter than a slow query, a retry storm against a struggling backend, or a keepalive that exhausts connections. Learning the timers and how they combine is the quickest way to stop guessing.

## What you will learn

- The timeouts on each leg of a request
- Keepalive to backends and to clients
- Retries and why they can make things worse
- Connection and file-descriptor limits

## The timers

```
proxy_connect_timeout 3s;     # time to open a connection to the backend
proxy_send_timeout    30s;    # gap between writes to the backend
proxy_read_timeout    30s;    # gap between reads from the backend
keepalive_timeout     65s;    # idle client connection
client_body_timeout   15s;    # slow upload protection
```

`proxy_read_timeout` is not the total time of a request; it is the maximum gap between two reads. A backend that sends nothing for 30 seconds is cut off even if the whole answer would have arrived at 31.

## Keepalive

Opening a new TCP (and TLS) connection per request is wasteful. Reuse backend connections:

```
upstream app_backend { server 10.0.1.11:8080; keepalive 32; }
location / { proxy_pass http://app_backend; proxy_http_version 1.1; proxy_set_header Connection "" \
    ; }
```

## Retries

`proxy_next_upstream error timeout;` retries another server on those failures. Every retry multiplies load: with 3 attempts, a struggling cluster sees up to three times the requests exactly when it can least cope. Keep retries few, limit them to safe methods and add a total time budget (`proxy_next_upstream_timeout`).

### Diagram: Timeouts must nest

| Part | What it does | What goes wrong |
|---|---|---|
| **User or client** | Waits up to its own timeout, for example 60 seconds. | If it times out first, the proxy keeps working for nobody. |
| **Proxy** | Has connect and read timeouts, for example 3 and 30 seconds. | A read timeout below the real response time cuts good requests. |
| **Application** | Calls a database with a 5 second timeout. | No timeout means a stuck thread holds a connection forever. |
| **Database** | Answers or fails within its own limit. | A slow query at the bottom becomes a 504 at the top. |
| **Retries** | Send a failed request to another backend. | Unbounded retries turn a slowdown into an outage. |

Flow: User or client to Proxy / Proxy to Application / Application to Database / Proxy to Retries

MARGA by EknathaLabs marga.eknathalabs.com

## Limits

Each connection uses a file descriptor. `worker_connections` times the number of workers caps concurrency, and each proxied request uses two connections (client and backend). Raise `worker_rlimit_nofile` and the OS limit together, and watch for "worker_connections are not enough" in the error log.

## Incident: The report that always timed out

**Incident:** *A report endpoint finished in 70 seconds but users always got a 504.*

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}\n' https://shop.example.com/reports/monthly
504 60.003
error.log: upstream timed out (110: Connection timed out) while reading response header from upstr \
    eam
```

The proxy gave up at 60 seconds while the app was still working. A longer timeout would hide a design problem, so the team made the report asynchronous: a request creates a job and the page polls for the result.

## Common mistakes

- Treating read timeout as the whole request time
- Setting the proxy timeout higher than the user's
- Retrying without a limit or a budget
- Raising worker_connections without raising the OS file limit

### Check yourself

**What does proxy_read_timeout measure?**

Answer: The gap between two reads from the backend

**Why can retries cause outages?**

Answer: They multiply load on a struggling backend

**What does keepalive to the backend save?**

Answer: New connection setup for every request

**What should timeouts do as you go up the stack?**

Answer: Grow, so callers wait longer than services they call

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 7. Caching, compression and static files

12 minutes. Serve static files efficiently, cache upstream responses with correct keys and expiry, and compress safely at the proxy.

The fastest request is one the backend never sees. A well-configured proxy cache can absorb most traffic for public content. A badly configured one leaks private pages. The difference is the cache key and the rules for what may be stored.

## What you will learn

- Serving static files with the right headers
- proxy_cache: key, validity, stale and locking
- What must never be cached
- Compression and its limits

## Static files

```
location /assets/ {
    root /var/www/shop;
    sendfile on;
    expires 30d;
    add_header Cache-Control "public, immutable";
}
```

Give assets a content hash in the file name (`app.3f9c1.js`) so they can be cached for a long time and replaced by changing the name, not by purging.

## Caching upstream responses

```
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:50m max_size=2g inactive=60m;
location /catalog/ {
    proxy_cache api;
    proxy_cache_key "$scheme$host$request_uri";
    proxy_cache_valid 200 301 5m;
    proxy_cache_valid 404 1m;
    proxy_cache_use_stale error timeout updating;
    proxy_cache_lock on;
    add_header X-Cache-Status $upstream_cache_status;
}
```

The **key** decides what counts as the same request. `proxy_cache_lock` makes concurrent requests for a missing item wait for one backend fetch, avoiding a stampede. `use_stale` serves an old copy while the backend is failing, which turns an outage into a slow patch. The header shows HIT, MISS, EXPIRED or STALE for debugging.

## What not to cache

Anything personalised (cookies, authorisation headers, account pages) must bypass the cache or vary the key by user. nginx does not cache responses with `Set-Cookie` or `Cache-Control: private` by default; leave those protections on.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: A cached request

| Part | What it does | What goes wrong |
|---|---|---|
| **User** | Asks for a public catalogue page. | A personalised page cached here leaks one user's data. |
| **Proxy cache** | Answers HIT from disk, or MISS and asks the backend. | A key that ignores a header or cookie mixes up different users. |
| **Backend** | Builds the page only on a MISS. | Without cache lock, a popular expiry sends a stampede here. |
| **Stale copy** | Served if the backend is slow or down. | Stale content can mislead; keep the window short. |
| **Compression** | Shrinks text responses before sending. | Compressing already-compressed files wastes CPU. |

Flow: User to Proxy cache / Proxy cache to Backend / Proxy cache to Stale copy / Proxy cache to Compression

## Compression

`gzip on; gzip_types text/css application/json application/javascript;` shrinks text by 70 percent or more. Do not compress images or video, which are already compressed, and keep levels modest (4 to 6): higher levels cost CPU for little gain.

## Incident: Everyone saw the same name

**Incident:** *A cached page that showed a user's name was served to others.*

```
$ curl -s -H 'Cookie: sid=bob' https://shop.example.com/home | grep Hello
Hello Alice
X-Cache-Status: HIT
```

The cache key was only the URL. The team bypassed the cache when a session cookie was present, moved the personal part of the page to a separate uncached request and added a test that fetches the page as two users.

## Common mistakes

- A cache key that ignores user-specific headers
- Long cache lifetimes on files with no content hash
- Caching error responses for too long
- Compressing images and archives

### Check yourself

**What decides whether two requests are the same in a cache?**

Answer: The cache key

**What does proxy_cache_lock help prevent?**

Answer: A stampede of identical backend fetches

**What should not be cached by default?**

Answer: Personalised responses

**What does X-Cache-Status HIT mean?**

Answer: The proxy answered from its cache

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 8. Rate limiting and protecting backends

13 minutes. Limit request and connection rates at the proxy, return correct status codes and protect backends from bursts and abusive clients.

Backends rarely fail from steady load; they fail from bursts and bugs. The proxy is the cheapest place to cap them, before a request costs a database query. A good limit is fair, visible and kind to well-behaved clients.

## What you will learn

- How the leaky bucket, burst and nodelay work
- Choosing the key: address, API key or token
- Returning 429 and Retry-After
- Limiting connections, body size and slow clients

## A request limit

```
limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;
limit_req_status 429;

location /api/ {
    limit_req zone=perip burst=20 nodelay;
    proxy_pass http://app_backend;
}
```

`rate=10r/s` fills the bucket at ten requests a second. `burst=20` lets a client go 20 requests over the rate before being refused; `nodelay` serves the burst immediately instead of spacing it out. Requests beyond are refused with 429.

## Who is counted

The key decides fairness. `$binary_remote_addr` counts per client address, which is wrong behind a CDN or NAT (many users, one address, or the CDN's address for everyone). Behind a CDN use the real-client header, validated from trusted proxies only. For APIs, key on an API key or token (`$http_x_api_key`) so each customer has a budget.

## Other limits

```
limit_conn_zone $binary_remote_addr zone=conn:10m;
limit_conn conn 20;           # at most 20 parallel connections per client
client_max_body_size 5m;      # large uploads rejected with 413
client_body_timeout 10s;      # slow uploads closed
```

Tell clients what happened: `429 Too Many Requests` with a `Retry-After` header lets well-written clients back off. Exempt health checks and internal addresses so monitoring does not get limited.

### Diagram: Where limits protect the backend

| Part | What it does | What goes wrong |
|---|---|---|
| **Clients** | Real users and the occasional buggy or abusive one. | One client in a retry loop can look like an attack. |
| **Rate limit** | Counts per key and refuses with 429 above the budget. | The wrong key limits innocent users sharing an address. |
| **Connection and body caps** | Cap parallel connections, upload size and slow senders. | Without them, slow clients tie up workers. |
| **Backend** | Sees only the traffic the proxy lets through. | Unprotected, a burst reaches the database. |
| **Metrics** | Count 429s per key and alert on spikes. | Silent limits hide a client bug or an attack. |

Flow: Clients to Rate limit / Rate limit to Connection and body caps / Connection and body caps to Backend / Rate limit to Metrics

MARGA by EknathaLabs marga.eknathalabs.com

## Incident: The retry loop

**Incident:** *A new app version retried failures instantly and flooded the API.*

```
access.log: 5200 req/s, 3900 distinct clients
upstream: 502 rate 41 percent
db: connections 100/100
```

The bug amplified every failure into more requests. A per-client limit with burst, 429 with Retry-After, and exponential backoff in the app stopped the loop and let the API recover.

## Common mistakes

- Limiting by address behind a CDN where every request has the CDN's address
- Rate limits with no metrics or alerts
- Limiting health checks and monitors
- No Retry-After so clients retry at once

### Check yourself

**What does burst allow?**

Answer: A short overshoot above the rate

**Which status do you return for rate limiting?**

Answer: 429

**Why is address-based limiting wrong behind a CDN?**

Answer: All users may share the CDN's address

**What does limit_conn control?**

Answer: Parallel connections per key

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 9. HAProxy: frontends, backends and ACLs

13 minutes. Configure HAProxy with frontends, backends, ACL-based routing, stick tables and health checks, and read its statistics.

HAProxy is built for balancing. Its configuration is explicit and its runtime tooling is excellent. Learning its four section types and how ACLs route lets you read any HAProxy file.

## What you will learn

- The sections of an HAProxy configuration
- ACLs and routing with use_backend
- Health checks, rise and fall
- The stats page and runtime socket

## Sections

```
global
    maxconn 20000
    log /dev/log local0
defaults
    mode http
    timeout connect 3s
    timeout client  30s
    timeout server  30s
    option httplog
frontend web
    bind :443 ssl crt /etc/haproxy/certs/
    acl is_api path_beg /api/
    acl is_admin hdr(host) -i admin.example.com
    use_backend api  if is_api
    use_backend admin if is_admin
    default_backend site
backend site
    balance roundrobin
    option httpchk GET /healthz
    server s1 10.0.1.11:8080 check inter 2s rise 2 fall 3
    server s2 10.0.1.12:8080 check inter 2s rise 2 fall 3
backend api
    balance leastconn
    server a1 10.0.2.11:9000 check
```

`global` is process-wide, `defaults` sets shared values, `frontend` accepts connections and `backend` holds servers. An **ACL** is a named test (path starts with /api/, host equals admin.example.com); `use_backend ... if` applies it. `rise 2 fall 3` means two good checks to enter rotation and three bad ones to leave, which avoids flapping.

## Live state

The stats page shows each server's status, sessions, queue and error counts. The **runtime socket** lets you change state without a reload:

```
$ echo "set server site/s1 state drain" | socat stdio /run/haproxy/admin.sock
$ echo "show servers state site" | socat stdio /run/haproxy/admin.sock
```

**Stick tables** store per-client counters (requests, errors) in memory, usable for rate limiting and stickiness.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How HAProxy routes a request

| Part | What it does | What goes wrong |
|---|---|---|
| **Frontend** | Accepts the connection and terminates TLS. | A wrong bind or certificate path stops all traffic. |
| **ACLs** | Name tests on host, path, header or source. | An ACL that is too broad sends traffic to the wrong backend. |
| **Backend** | Chooses a server using the balance method and health. | If all servers are down, requests get 503. |
| **Server** | Answers the request. | A slow server fills the queue; watch queue depth. |
| **Stats and runtime socket** | Show live status and allow drain without reload. | Expose it only to admins; it can change state. |

Flow: Frontend to ACLs / ACLs to Backend / Backend to Server / Backend to Stats and runtime socket

## Incident: The 503 on every request

**Incident:** *After a deploy all servers failed health checks and HAProxy answered 503 for every request.*

```
$ echo "show servers state site" | socat stdio /run/haproxy/admin.sock
site s1 10.0.1.11 srv_op_state=0 (L7STS 404)
site s2 10.0.1.12 srv_op_state=0 (L7STS 404)
haproxy: backend site has no server available!
```

The new release moved the health endpoint from /health to /healthz, so the check got 404 and marked every server down. The fix was to update the check path, and the team added the health path to the release checklist and a canary server.

## Common mistakes

- Health check path that differs from the real endpoint
- Setting fall to 1, which causes flapping
- Exposing the admin socket or stats page publicly
- Reloading for a change the runtime socket can make

### Check yourself

**What does an ACL do?**

Answer: Names a test used for routing decisions

**What does rise 2 fall 3 mean?**

Answer: Two good checks to enter, three bad to leave

**How can you drain a server without reload?**

Answer: Use the runtime socket to set state drain

**What does a stick table store?**

Answer: Per-client counters in memory

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 10. Envoy, Traefik and Caddy: dynamic proxies

13 minutes. Compare proxies that configure themselves from service discovery: Envoy listeners, routes and clusters, Traefik providers and Caddy automatic HTTPS.

In containers and Kubernetes, backends appear and disappear all day. Static files and reloads work, but several proxies were built to follow the platform instead. They differ in complexity, features and who they suit.

## What you will learn

- Envoy: listeners, routes, clusters and endpoints
- Traefik: entrypoints, routers, services and providers
- Caddy: automatic HTTPS and a short config
- How to choose between nginx, HAProxy, Envoy, Traefik and Caddy

## Envoy

Envoy is a high-performance proxy with rich observability, used as the data plane in many service meshes and gateways. Its model: a **listener** accepts connections, **filters** (such as the HTTP connection manager) process them, **routes** map requests to a **cluster**, and a cluster holds **endpoints**. The control plane sends updates over **xDS** APIs, so routes and endpoints change live.

```
static_resources:
  listeners:
  - name: main
    address: { socket_address: { address: 0.0.0.0, port_value: 8080 } }
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3 \
    .HttpConnectionManager
          stat_prefix: ingress
          route_config:
            virtual_hosts:
            - name: all
              domains: ["*"]
              routes:
              - match: { prefix: "/api" }
                route: { cluster: api }
          http_filters:
          - name: envoy.filters.http.router
            typed_config: { "@type": type.googleapis.com/envoy.extensions.filters.http.router.v3.R \
    outer }
```

## Traefik

Traefik discovers services from **providers** (Docker labels, Kubernetes Ingress or IngressRoute, files). An **entrypoint** is a port, a **router** matches rules (`Host(...)`, `PathPrefix(...)`), and a **service** lists the servers. Middleware adds rate limits, auth and redirects.

```
labels:
  - "traefik.http.routers.shop.rule=Host(`shop.example.com`)"
  - "traefik.http.routers.shop.tls.certresolver=letsencrypt"
  - "traefik.http.services.shop.loadbalancer.server.port=8080"
```

## Caddy

Caddy obtains and renews HTTPS certificates by default and its config is short:

```
shop.example.com {
    reverse_proxy 10.0.1.11:8080 10.0.1.12:8080
    encode gzip
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

## Choosing

nginx: very common, great for web serving and caching. HAProxy: excellent load balancing and live state. Envoy: service meshes, gRPC, deep metrics. Traefik: containers and Kubernetes with automatic discovery. Caddy: small deployments and simple HTTPS.

### Diagram: A dynamic proxy follows the platform

| Part | What it does | What goes wrong |
|---|---|---|
| **Platform** | Containers or pods start and stop; labels or resources describe them. | Wrong labels mean a service is invisible to the proxy. |
| **Discovery** | The provider or xDS control plane watches for changes. | A control plane outage freezes routes at the last known state. |
| **Proxy** | Updates routes live without a reload. | A bad update can send traffic nowhere; validate before applying. |
| **Services** | Receive traffic as soon as they are ready. | Routing before readiness gives errors during a rollout. |
| **Metrics and access logs** | Show routes, latency and errors per service. | Without them, a dynamic proxy is a black box. |

Flow: Platform to Discovery / Discovery to Proxy / Proxy to Services / Proxy to Metrics and access logs

## Incident: The route nobody could see

**Incident:** *A new service never received traffic because its label had a typo.*

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://new.example.com
404
traefik dashboard: no router matches host new.example.com
label: traefik.http.routers.neww.rule  (typo: neww)
```

The label mentioned a router name that did not match the service definition, so no route existed. The team added a dashboard check to the deploy pipeline and a smoke test that requests the new host.

## Common mistakes

- Applying dynamic configuration with no validation or smoke test
- Exposing the proxy dashboard or admin API publicly
- Using Envoy for a simple site where nginx or Caddy is enough
- Not monitoring the control plane that feeds the proxy

### Check yourself

**What is xDS in Envoy?**

Answer: APIs that deliver configuration updates live

**Which proxy gets certificates automatically by default?**

Answer: Caddy

**What does a Traefik provider do?**

Answer: Discovers services from a platform

**What is a good way to check a dynamic route after deploy?**

Answer: A smoke test requesting the new host

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 11. Zero-downtime reloads and blue-green at the proxy

12 minutes. Change proxy configuration and shift traffic between versions without dropping requests, using graceful reloads, weighted routing and drain.

The front door should never be closed for maintenance. Reloads, weights and drains let you change both the proxy and the services behind it while users keep working.

## What you will learn

- What a graceful reload does
- Blue-green switching and rollback
- Canary routing by weight, header or cookie
- Draining and readiness during deploys

## Graceful reload

`nginx -s reload` (or `systemctl reload nginx`) parses the new config. If it is valid, new workers start with it and old workers stop accepting new connections and exit when their current requests finish. If the new config is invalid, the old workers keep running. A **restart** would drop everything, so use reload for changes.

## Blue-green

Run two complete environments, blue (live) and green (new). Test green privately, then switch the proxy's upstream in one reload. Rollback is switching back.

```
upstream app_live { server 10.0.1.11:8080; server 10.0.1.12:8080; }  # blue
upstream app_next { server 10.0.2.11:8080; server 10.0.2.12:8080; }  # green
map $http_x_canary $pool { default app_live; "1" app_next; }
server { location / { proxy_pass http://$pool; } }
```

The header route lets testers reach green through the real proxy before anyone else does.

## Canary by weight

```
split_clients "${remote_addr}${http_user_agent}" $pool {
    5%  app_next;
    *   app_live;
}
```

Send 5 percent to the new version, watch error rate and latency, then 25, 50 and 100. Roll back by setting the share to zero.

### Diagram: Shifting traffic safely

| Part | What it does | What goes wrong |
|---|---|---|
| **Blue (live)** | The current version serving everyone. | It must stay healthy until the new version is proven. |
| **Green (new)** | The new version, tested privately first. | An untested green receives real users at the switch. |
| **Proxy** | A reload switches or weights the upstream. | Restarting instead of reloading drops connections. |
| **Error rate and latency** | Compare the new version with the old at each step. | No comparison means no signal to roll back. |
| **Rollback** | Return the share to blue in one reload. | A rollback that was never tested may fail when needed. |

Flow: Proxy to Blue (live) / Proxy to Green (new) / Proxy to Error rate and latency / Error rate and latency to Rollback

MARGA by EknathaLabs marga.eknathalabs.com

## Drain and readiness

Remove a server from rotation, wait for current requests to finish, then stop it. A new server should join only after its readiness check passes. In Kubernetes this is what readiness probes and preStop hooks do for you; on plain hosts you do it with the proxy's drain or weight controls.

## Incident: The restart that dropped uploads

**Incident:** *A config change applied with a service restart cut every open connection during business hours.*

```
$ systemctl restart nginx
access.log: 1042 requests ended with status 499
client: net::ERR_CONNECTION_RESET
```

Users lost uploads in progress. The team switched to nginx -t and reload for every change, and moved risky changes to a canary share first. Restart is reserved for upgrades, and happens only after drain.

## Common mistakes

- Restarting instead of reloading
- Switching all traffic at once with no way back
- Stopping a server before its requests finish
- Sending traffic to a server before it is ready

### Check yourself

**What happens to old workers on reload?**

Answer: They finish current requests and exit

**What is blue-green?**

Answer: Two environments with a switch between them

**What does a canary share do?**

Answer: Sends a small part of traffic to the new version first

**What should happen before stopping a server?**

Answer: Drain it

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 12. Debugging 502, 504 and other proxy problems

14 minutes. Diagnose proxy errors in a fixed order using status codes, error logs, upstream timing and direct requests to the backend.

Proxy errors feel mysterious because the user, the proxy and the backend each see a different failure. A short method separates them: look at what the proxy logged, repeat the request directly to the backend, and compare.

## What you will learn

- What 502, 503, 504 and 499 each point to
- Logging upstream status and timing
- Repeating the request around the proxy
- Common causes: DNS, ports, timeouts, buffers, limits

## Read the status code

| Code | Meaning | First place to look |
|---|---|---|
| 400 / 413 / 431 | Bad request, body or headers too large | Proxy limits, `client_max_body_size`, header buffers |
| 499 | Client closed the connection | Client timeouts, slow responses |
| 502 | Bad gateway: no valid reply from upstream | Is the backend up, right port, crash, wrong protocol |
| 503 | No upstream available, or overload | Health checks, rate limits, backend capacity |
| 504 | Gateway timeout | Slow backend, timeout settings |

## Log what matters

```
log_format main '$remote_addr "$request" $status '
                'rt=$request_time urt=$upstream_response_time us=$upstream_status ua=$upstream_add \
    r';
access_log /var/log/nginx/access.log main;
```

`urt` is the backend's time, `rt` the total. If `rt` is high but `urt` is low the delay is the client or the proxy; if both are high the backend is slow. `ua` names which server answered.

## Go around the proxy

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}\n' http://10.0.1.11:8080/healthz
200 0.004
$ curl -s -o /dev/null -w '%{http_code} %{time_total}\n' https://shop.example.com/api/orders
502 0.002
$ tail -2 /var/log/nginx/error.log
connect() failed (111: Connection refused) while connecting to upstream, upstream: "http://10.0.1. \
    12:8080/api/orders"
```

If the direct request works and the proxied one fails, compare ports, hostnames, protocol (http versus https), headers and the upstream address in the error log. A fast 502 usually means connection refused; a slow 504 means the upstream was reached but did not answer.

## Frequent causes

- **Stale DNS:** nginx resolves upstream hostnames at startup. Use a resolver and a variable in proxy_pass, or reload after changes.
- **Wrong scheme or port:** speaking HTTP to an HTTPS backend gives 502 with an SSL error in the log.
- **Backend crash or restart loop:** check its logs and restart count.

MARGA by EknathaLabs marga.eknathalabs.com

- **Header too large:** big cookies cause 400 or 502; raise `large_client_header_buffers` or shrink the cookie.
- **Limits:** file descriptors, worker connections or backend pool exhausted.

### Diagram: Narrowing a proxy error

| Part | What it does | What goes wrong |
|---|---|---|
| **Status seen by the user** | 502, 503, 504, 499 or 4xx tells which side to suspect. | The same page can be a different cause each time. |
| **Proxy logs** | error.log reason and access.log upstream status and timing. | Missing upstream fields leave you guessing. |
| **Request the backend directly** | Bypass the proxy to test the backend alone. | A direct test from another network may behave differently. |
| **Cause** | DNS, port, protocol, timeout, header size or capacity. | Changing several settings hides which one mattered. |
| **Fix and verify** | Change one thing and repeat the same request. | Without a repeat test the fix is only a guess. |

Flow: Status seen by the user to Proxy logs / Proxy logs to Request the backend directly / Request the backend directly to Cause / Cause to Fix and verify

## Incident: The 502 that appeared after a failover

> **Incident:** *A database failover moved a backend to a new IP, and the proxy kept sending to the old address.*

```
error.log: connect() failed (113: No route to host) while connecting to upstream, upstream: "http: \
    //10.0.1.12:8080/"
$ dig +short app.internal
10.0.1.30
```

nginx had resolved the name once at startup. A reload fixed it at once, and the long-term fix was to use a resolver with a short valid time and a variable in proxy_pass so the name is looked up again.

## Common mistakes

- Debugging from the browser instead of the proxy log
- Not logging upstream status and timing
- Changing timeouts before knowing which side is slow
- Forgetting that nginx caches upstream DNS names

### Check yourself

**What does a fast 502 with "Connection refused" suggest?**

Answer: Nothing listens on the upstream port

**What does 504 point to?**

Answer: The upstream was reached but too slow

**What does status 499 in nginx logs mean?**

Answer: The client closed the connection

**Why log $upstream_response_time?**

Answer: To separate backend time from total time

MARGA by EknathaLabs marga.eknathalabs.com

# Cheat sheet: Reverse proxies and load balancers

nginx and HAProxy basics, the status codes you will see, timeouts, health checks and a debugging order for 502 and 504.

## Test and reload safely

```
nginx -t                      # syntax and file check, never reload without it
nginx -T | less               # print the full effective configuration
nginx -s reload               # graceful reload, old workers finish their requests
tail -f /var/log/nginx/error.log
```

## A minimal reverse proxy

```
upstream app {
    server 10.0.2.14:8080 max_fails=3 fail_timeout=10s;
    server 10.0.2.15:8080;
    keepalive 32;
}
server {
    listen 443 ssl;
    server_name shop.example.com;
    location / {
        proxy_pass http://app;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 3s;
        proxy_read_timeout 30s;
    }
}
```

## Status codes from the proxy

| Code | Meaning | First check |
|---|---|---|
| 502 | No valid answer from the upstream | Error log: connection refused or reset |
| 503 | No healthy upstream or limit reached | Health checks, connection limits |
| 504 | Upstream too slow | Backend latency, database, timeouts |
| 429 | Rate limit applied | Limit settings and client behaviour |

## Look from outside and inside

```
curl -sI https://shop.example.com | head -5
curl -s -o /dev/null -w '%{http_code} %{time_connect} %{time_starttransfer}\n' https://shop.exampl \
    e.com/
ss -ltnp | grep -E ':80|:443|:8080'
openssl s_client -connect shop.example.com:443 -servername shop.example.com </dev/null | openssl x \
    509 -noout -dates
```

## HAProxy essentials

```
haproxy -c -f /etc/haproxy/haproxy.cfg     # validate configuration
echo "show stat" | socat stdio /run/haproxy/admin.sock | cut -d, -f1,2,18
```

## Debugging order

1. Read the error log line and note the upstream address and error text.
2. Connect to the backend from the proxy host with curl.
3. Check backend health, CPU, database and connection pools.
4. Review the last change: deploy, configuration, certificate.
5. Turn off risky retries, add limits, fix the slow dependency, then tune timeouts.

MARGA by EknathaLabs
marga.eknathalabs.com

## Safe changes

```
nginx -t && nginx -s reload
```

```
curl -sv -H 'Host: app.example.com' http://127.0.0.1/ -o /dev/null
```

```
curl -sI https://app.example.com | grep -i 'strict-transport\|server'
```

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

**Warning:** *Always run* `nginx -t` *before a reload. A bad config makes the reload fail, and a restart instead of a reload drops live connections.*

**Warning:** *A trailing slash in* `proxy_pass` *changes the path sent to the backend. Test one URL by hand after every change.*

**Danger:** *Never cache responses that carry cookies or* `Authorization` *headers without a key that separates users. One user can see another user's page.*

**Warning:** *Retrying non-idempotent requests (POST) on a timeout can charge or create twice. Limit* `proxy_next_upstream` *to safe methods.*

MARGA by EknathaLabs
marga.eknathalabs.com